PHANTOM PROTOCOL

Technical Specification № 4

Composability and Formal Security Model

UC Ideal Functionalities · Subsystem Interactions · Bounded Guarantees · Open Assumptions

March 2026 · Draft v0.1

ABSTRACT

This specification defines the formal security model for Phantom Protocol in its entirety. It provides: (1) a precise adversary model distinguishing four adversary classes — Passive, Active, Nation-State, and Quantum — and the guarantees the protocol provides against each; (2) Universal Composability (UC) ideal functionality definitions for each of the five protocol subsystems — the Trustless PKI (F^{PKI}), the Verifiable Mix Network (F^{MIX}), the Hidden Service Layer (F^{HS}), the DC-Net Broadcast Layer (F^{DCN}), and the Cover Traffic Engine (F^{COV}); (3) a composability analysis identifying the three interaction points where subsystem boundaries leak security-relevant information; (4) explicit, quantified bounds on the anonymity guarantees in the standard mix mode, including the per-epoch guarantee, the multi-epoch degradation curve, and the minimum viable anonymity set k^{min}; (5) the trust assumptions that underpin each guarantee and the open assumptions that remain unresolved; and (6) the conditions under which each guarantee fails. This document is the prerequisite for the Phase 5 external security audit and constitutes the authoritative statement of what Phantom Protocol can and cannot prove.

Table of Contents

1. Purpose and Scope of This Specification
2. Notation and Mathematical Conventions
3. Adversary Model — Four Classes with Distinct Capability Profiles
4. Universal Composability Framework — Approach and Adaptations
5. F_{PJT} — Ideal Functionality: Trustless PKI
6. F^{Mix} — Ideal Functionality: Verifiable Mix Network
7. F^{Hs} — Ideal Functionality: Hidden Service Layer
8. F^{Dcn} — Ideal Functionality: DC-Net Broadcast Layer
9. F^{cov} — Ideal Functionality: Cover Traffic Engine
10. Subsystem Composability — The Three Interaction Leaks
11. Quantified Anonymity Bounds
12. Trust Assumption Register
13. Guarantee Failure Conditions
14. Open Assumptions and Research Questions
15. Relationship to Other Specifications
16. References

1. Purpose and Scope of This Specification

All previous Phantom Protocol specifications define what each subsystem does. This specification defines what each subsystem proves, what it assumes, where those assumptions can be violated, and what happens to the overall protocol when they are.

The motivation is the non-composability problem identified in the security audit (GAP-04): five subsystems — each with individually argued security properties — have not been formally analysed in combination. The canonical failure mode of composed cryptographic systems is that individually secure components produce insecure interactions. This specification closes that gap.

Scope: This document covers the entire Phantom Protocol stack as defined in Specifications № 1 through № 3 and the Whitepaper. It does not specify implementation behaviour — that remains in the component specifications. It does not specify cryptographic primitives — those are defined in FIPS 203, FIPS 204, and the referenced academic literature. It specifies the security model and composability claims that must hold for the protocol to satisfy its stated anonymity goals.

Non-scope: Transport-layer obfuscation (the QUIC mimicry layer) and network-level adversaries operating below the Phantom protocol boundary (BGP hijacking, AS-level observation) are outside the formal model. These are addressed in the security audit and in implementation guidelines, but they do not appear in the UC ideal functionalities because they operate at a different abstraction layer.

> ⚠ **Mandatory Reading Gate**
>
> No Phase 1 implementation code should be merged before this specification has been reviewed by an external cryptographer. The formal model defined here constrains the implementation — not the reverse. Design decisions made during implementation that conflict with the guarantees in this document must trigger a specification revision, not a model weakening.

2. Notation and Mathematical Conventions

The following notation is used consistently throughout this specification.

Symbol	Meaning
λ	Security parameter. All algorithms run in time poly(λ). Default: $\lambda = 256$.
negl(λ)	A function f such that for all polynomials p, $f(\lambda) < 1/p(\lambda)$ for all sufficiently large λ.
PPT	Probabilistic Polynomial Time. An adversary A is PPT if it runs in time poly(λ).
F^x	Ideal functionality for subsystem x. A trusted party that implements x perfectly.
Π^x	Real-world protocol for subsystem x. What the actual implementation computes.
EXEC(Π, A, Z)	The output of environment Z when interacting with adversary A attacking protocol Π.
IDEAL(F, S, Z)	The output of Z when interacting with simulator S in the ideal world with functionality F.
Π UC-realises F	Π securely realises F if for every PPT A there exists a PPT S such that for all Z: $\lvert \Pr[\mathrm{EXEC}(\Pi,A,Z)=1] - \Pr[\mathrm{IDEAL}(F,S,Z)=1] \rvert \leq \mathrm{negl}(\lambda)$.
k	Anonymity set size (number of indistinguishable senders in a given epoch).
k^{mdn}	Minimum viable anonymity set (below which the protocol refuses sensitive routing).
N	Total number of active Phantom nodes in the current epoch.
f	Fraction of nodes controlled by the adversary. Protocol assumes $f < 1/2$.
T	Number of elapsed epochs. Used in multi-epoch intersection attack bounds.
ε_t	Per-epoch deanonymisation probability against a given adversary class.
B	Batch size (number of packets per mix round). Default: $32 \leq B \leq 512$.
H(·)	BLAKE3 hash function, modelled as a random oracle in proofs.
C^{In}, C^{out}	Pedersen commitment vectors over the input and output of a mix batch.
π_s^h	ZK shuffle proof for batch b, produced by a mix node.
σ	Secret shuffle permutation (known only to the mix node).
pk^{LIM}	Long-term post-quantum KEM public key (Kyber-1024) for node i.
sk^{LIM}	Long-term post-quantum KEM secret key for node i.
addr(.phantom)	A .phantom hidden service address: BLAKE3($pk^{ed} \parallel pk^{DIL}$).
CRS	Common Reference String for the BLS12-381 Pedersen commitment scheme.
E_n	Epoch number n. Each epoch lasts 3600 seconds.
R	Rendezvous node (in the hidden service protocol).
IP^i	Introduction point i (in the hidden service protocol).

Table 1 — Notation reference for this specification

3. Adversary Model — Four Classes with Distinct Capability Profiles

The whitepaper describes a single 'Global Passive Adversary' threat model. This is a useful high-level framing but is insufficiently precise for formal security analysis. Different subsystems provide different guarantees against different adversary classes. This section defines four adversary classes precisely, so that each guarantee in §§5–9 can be tied to the adversary class against which it holds.

3.1 Class-I Adversary: Passive Network Observer

Capabilities: Observes all packet metadata on all network links simultaneously. Records timing, size, volume, and endpoint information for all Phantom QUIC flows. Cannot modify packets, cannot control any node, cannot perform legal compulsion. Has unlimited storage and computation time for retrospective analysis.

Cannot: Decrypt any packet payload. Observe packet contents. Control any Phantom node. Inject or drop packets.

Realistic example: An ISP or internet exchange point performing passive traffic analysis. A nation-state operating a passive collection programme at internet backbone level.

Protocol guarantee against Class-I: Sender anonymity with anonymity set size $k \geq k^{min}$ per epoch. Receiver anonymity. Relationship unlinkability within a single epoch. Cover traffic renders timing correlation non-viable within the batch window. Formally: no Class-I adversary can identify a sender with probability exceeding $1/k + negl(\lambda)$.

3.2 Class-II Adversary: Active Network Adversary (Malicious Relay Operator)

Capabilities: All Class-I capabilities, plus: controls up to $f < 1/3$ of Phantom relay nodes (the $f < 1/3$ bound is derived from the S/Kademlia eclipse resistance analysis at d=5, quorum=3, as cited in Spec № 3). Controlled nodes follow the protocol honestly but record all observable information at those nodes. Controlled nodes may attempt to produce valid ZK shuffle proofs while recording which input became which output. Controlled nodes may attempt to correlate ingress and egress packet timing.

Cannot: Forge ZK shuffle proofs for inputs that were not honestly processed. Forge node identities (self-certifying construction). Publish false NodeDescriptors that pass signature verification. Compel nodes it does not control.

Realistic example: A well-funded organisation (private or state) that has infiltrated a fraction of the volunteer relay pool. An adversary running a large set of honest-but-recording Sybil nodes that passed the PoW admission process.

Protocol guarantee against Class-II: Sender anonymity holds as long as not all hops on a sender's circuit are controlled adversarial nodes. The ZK shuffle proof prevents a controlled node from silently reordering packets — any attempt to do so produces a detectable invalid proof. End-to-end confidentiality holds unconditionally (no honest node decrypts application payload). Formally: the probability that all three hops of a circuit are controlled is at most $f^3 = (1/3)^3 \approx 3.7\%$.

3.3 Class-III Adversary: Nation-State / Legal Compulsion Adversary

Capabilities: All Class-II capabilities, plus: can legally compel any identifiable legal entity (node operators in accessible jurisdictions, software distributors, certificate authorities, DNS operators). Has access to GPU/ASIC compute farms enabling Sybil attacks at 100–1000x the cost modelled in Spec № 3's CPU-based estimates. Has persistent long-term presence: can observe traffic for months to years and execute intersection attacks across many epochs. Has access to BGP-level routing manipulation. May attempt to perform a BLS12-381 trusted setup compromise if the CRS ceremony was insufficiently distributed.

Cannot (current assumptions): Break NIST-standardised post-quantum primitives (ML-KEM-1024, ML-DSA). Break the BLAKE3 random oracle assumption. Control a majority ($f \geq 1/2$) of Phantom nodes. Force a user's device to reveal keys without physical access.

Realistic example: The primary target adversary for Phantom Protocol. NSA-class global surveillance infrastructure. The GCHQ/Five Eyes collection programmes. The Great Firewall operating in active block-and-identify mode.

Protocol guarantee against Class-III: This is where the protocol's guarantees weaken significantly. Per-epoch sender anonymity holds with $k \geq k^{min}$. Long-term (multi-epoch) anonymity degrades at rate ε_t per epoch (see §11 for the quantified bound). The ZK shuffle proof prevents traffic deanonymization via relay compromise, but cannot prevent intersection attacks by a long-term observer. Cover traffic prevents single-epoch timing correlation. The predecessor attack (no entry guards) means that over T epochs, the probability that a Class-III adversary has been the first relay at least once approaches $1 - (1 - f)^T$. For $f = 0.2$ and $T = 50$ circuits: 99.999%.

> ⚠ **Honest Assessment of Class-III Guarantee**
>
> Phantom Protocol does NOT provide long-term anonymity against a Class-III adversary as currently specified. The intersection attack, the predecessor attack, and the GPU-enabled Sybil attack together mean that a nation-state adversary with persistent access can deanonymize users over weeks to months. The DC-Net extreme mode provides information-theoretic anonymity within a group but does not address the predecessor attack on the path to the DC-Net group. This is an honest statement of scope, not a design failure — it is the current state of the art. No deployed system provides long-term Class-III anonymity.

3.4 Class-IV Adversary: Quantum Adversary

Capabilities: All Class-III capabilities, plus: access to a cryptanalytically relevant quantum computer (CRQC) capable of running Shor's algorithm against RSA-2048, ECDSA, and X25519/X448 in feasible time. Cannot break ML-KEM-1024 or ML-DSA (NIST security level 3, equivalent to AES-192 classical security against quantum adversaries).

Cannot: Break ML-KEM-1024 (post-quantum KEM). Break ML-DSA (post-quantum signatures). Break AES-256 or ChaCha20-Poly1305 (symmetric, quantum-resistant at $\lambda/2$ security). Break BLAKE3 as a collision-resistant hash or random oracle.

Protocol guarantee against Class-IV: Session confidentiality is maintained via the hybrid construction (X25519 + Kyber-1024). The session key requires breaking both; a CRQC breaks X25519 but not Kyber-1024. Node identity is protected by Dilithium-3; classical Ed25519 can be forged by a CRQC, but the Dilithium-3 layer remains binding. Forward secrecy is maintained because per-epoch KEM keys are deleted after epoch end — even if the CRQC can break the archived Kyber key material, the session keys were derived ephemerally and are gone. Note: the CRIT-01 finding (Kyber fragment scheme, unaudited) is specifically a concern for the Class-

IV guarantee; until the fragment scheme is proven, the Kyber layer should be treated as untrusted.

4. Universal Composability Framework — Approach and Adaptations

The security definitions in §§5–9 use the Universal Composability (UC) framework of Canetti (2001). This section explains the framework and the adaptations made for the Phantom Protocol context.

4.1 Why UC

UC is chosen over game-based security definitions for two reasons. First, UC security is composable by construction: if each subsystem UC-realises its ideal functionality, then any composition of those subsystems is secure against any adversary, even one that attacks multiple components simultaneously. This directly addresses the non-composability risk (GAP-04). Second, UC ideal functionalities can be stated precisely without reference to the protocol that realises them, making them useful as both a specification and an audit target.

4.2 Adaptations

Network model: The UC framework assumes an authenticated and point-to-point network between parties. Phantom Protocol operates over an adversarial network where the adversary can observe all traffic (Class-I and above). We use the *UC with global setup* extension (Canetti et al., 2007) to model the global network observation capability as a shared reference functionality F^{net} available to all parties and the adversary simultaneously.

Asynchronous communication: The standard UC model assumes synchronous rounds. Phantom Protocol is asynchronous (batches form on timing, not on explicit rounds). We use the *asynchronous UC* variant (Backes, Pfitzmann, Waidner, 2004) in which the adversary controls message delivery timing within a bounded window. For Phantom, this bound is the batch window (700ms).

Adaptive vs. static corruption: Phantom's adversary model (Class-II and above) assumes *adaptive corruption* — the adversary can choose which nodes to corrupt after seeing the protocol run. All ideal functionalities below are stated for the adaptive corruption model. This is the stronger (harder to realise) assumption.

Quantum UC: For Class-IV guarantees, we use the *quantum UC* model (Unruh, 2010), which models the simulator as a quantum polynomial-time algorithm. The hybrid construction (X25519 + Kyber-1024) is analysed in the QROM (quantum random oracle model).

The composition theorem: Under UC, if Π^{PKI} UC-realises F^{PKI}, Π^{MIX} UC-realises F^{MIX} in the F^{PKI}-hybrid model, and so on for each subsystem, then the full Phantom Protocol stack UC-realises the composed ideal functionality $F^{Phantom} = F^{PKI} \circ F^{MIX} \circ F^{HS} \circ F^{DCN} \circ F^{COV}$. This is the composability theorem. Proving each component in the hybrid model of its dependencies is the proof strategy. Critically, this theorem holds only if the interaction leaks identified in §10 are correctly handled.

4.3 Current Status of Each UC Proof

Subsystem	UC Proof Status	Blocking Issue	Phase Target
F_{PJT} (Trustless PKI)	Sketch — argued informally in Spec № 3	CRIT-02: CRS ceremony required before formal proof	Phase 2
F^{Mix} (Verifiable Mix)	Not yet started	CRIT-01: Kyber fragment unproven; CRIT-02: CRS required	Phase 2
F^{Hs} (Hidden Services)	Not yet started	Depends on F_{PJT} and F^{Mix} being proven first	Phase 4
F^{Dcn} (DC-Net)	Not yet started	GAP-03: Pad establishment protocol undefined; GAP-05: Liveness	Phase 4
F^{cov} (Cover Traffic)	Not yet started	GAP-04: Interaction with F^{Mix} timing is unanalysed	Phase 3
Full composition	Not yet started	All component proofs must exist first	Phase 5 gate

Table 2 — UC proof status by subsystem. Each is a prerequisite for the Phase 5 audit.

5. F_{PJT} — Ideal Functionality: Trustless PKI

F_{PKI} is the ideal trusted party that replaces the Phantom-DPKI layer (Spec № 3). In the ideal world, it correctly maintains a directory of active nodes, provides self-certifying identities, and rejects Sybil identities without any of the real-world vulnerabilities of the Kademlia DHT.

5.1 Functionality Definition

```
Functionality F_PKI
==============================

Parties:  N honest nodes {P_1, ..., P_n}, adversary A, environment Z.

State:
  Directory  : Map<NodeId, NodeDescriptor>   // active nodes
  BanList    : Set<NodeId>                   // ejected nodes
  Epoch      : u32                           // current epoch

On (REGISTER, pk_ed, pk_dil, pk_x25519, pk_kyber, admission_token) from P_i:
  node_id := H("phantom-node-v1" || pk_ed || pk_dil || pk_x25519 || pk_kyber)
  If VerifyAdmissionToken(node_id, admission_token, Epoch) = false:
    Return (REJECT, "invalid_admission")             // Sybil resistance
  If node_id in BanList:
    Return (REJECT, "banned")
  Directory[node_id] := NodeDescriptor { pk_ed, pk_dil, pk_x25519, pk_kyber,
                                         epoch: Epoch, ... }
  Return (REGISTERED, node_id)

On (LOOKUP, target_id) from any party:
  If target_id in Directory and Directory[target_id].epoch = Epoch:
    Return (FOUND, Directory[target_id])             // exact, no eclipse
  Else:
    Return (NOT_FOUND)

On (REPORT, offender_id, evidence) from P_i:
  If VerifyEvidence(offender_id, evidence) = true:
    BanList.insert(offender_id)
    Directory.remove(offender_id)
    Notify all parties: (EJECTED, offender_id)       // instant propagation

On (EPOCH_TICK) from the scheduler:
  Epoch := Epoch + 1
  Remove all descriptors with descriptor.epoch < Epoch - 1
```

What F_{PKI} guarantees that the real protocol cannot: (1) Lookup always returns the correct descriptor or NOT_FOUND — no eclipse. (2) Ejection is instant and global — no gossip propagation delay. (3) Sybil resistance is perfect — no GPU/ASIC bypass. The real protocol Π_{PKI} UC-realises F_{PKI} only under specific assumptions listed in §12.

5.2 Self-Certification Security Property

The self-certifying identity construction (node_id = H(pubkeys)) provides the following formally statable property:

```
Definition: Self-Certification
A NodeDescriptor D is self-certifying if:

  Verify(D) = 1  iff
    H("phantom-node-v1" || D.pk_ed || D.pk_dil || D.pk_x25519 || D.pk_kyber)
    = D.node_id
    AND Ed25519.Verify(D.pk_ed, D.canonical_bytes(), D.sig_ed) = 1
    AND Dilithium3.Verify(D.pk_dil, D.canonical_bytes(), D.sig_dil) = 1
    AND |D.epoch - current_epoch()| <= 1

Theorem (Unforgeability): Under the BLAKE3 random oracle assumption and the
unforgeability of Dilithium-3 (EUF-CMA), no PPT adversary can produce a
NodeDescriptor D' such that Verify(D') = 1 and D'.node_id was not generated
by the honest key generation procedure, except with probability negl(lambda).

Proof sketch: D'.node_id = H(pk_ed' || ...) binds D' to specific public keys.
Forging a valid D' with a different node_id from the pubkeys requires either
(a) finding a BLAKE3 collision (negl by RO assumption) or (b) forging a
Dilithium-3 signature without the secret key (negl by EUF-CMA of Dilithium-3).
```

5.3 Admission Token Binding

The dual-puzzle admission token provides the following binding property, under the assumption that BLAKE3 is a random oracle:

```
Static puzzle binding (C1 = 20 bits):
  Pr[A finds (sk, pk) s.t. BLAKE3(pk)[0..C1] = 0...0] <= 2^{-C1} * Q_hash
  where Q_hash is the number of hash queries A makes.
  For C1 = 20: upper bound = Q_hash / 2^{20}.
  For Q_hash = 2^{40} (GPU farm): Pr <= 2^{20} = ~1M valid identities.

Dynamic puzzle epoch-binding (C2 = 16 bits):
  For each epoch E, find nonce s.t. BLAKE3(node_id XOR nonce || E)[0..C2] = 0
  Cost per epoch per identity: 2^{C2} hash evaluations.
  Pre-computation across epochs is impossible: each puzzle is epoch-specific.

NOTE: These bounds assume CPU-level BLAKE3 throughput (~2x10^8 H/s).
With GPU acceleration (~2x10^{10} H/s), the effective C1 bound drops by ~100x.
See MED-02 (audit) for the recommended Argon2id replacement.
```

6. F_{MIX} — Ideal Functionality: Verifiable Mix Network

F_{MIX} is the ideal mixing party that takes input packets from senders, permutes them undetectably, decrypts one Sphinx layer, and delivers them to the next hop — with perfect verifiability and zero information leakage about the permutation. It operates in the F_{PKI}-hybrid model (it can call F_{PKI} to look up node keys).

6.1 Functionality Definition

```
Functionality F_MIX  (in F_PKI-hybrid model)
=============================================

State:
  BatchQueue  : Queue<Packet>
  BatchWindow : Duration = 200ms
  ProofStore  : Map<BatchId, ShuffleProof>

On (SEND, packet, next_layer) from sender S:
  If packet.epoch != current_epoch(): Return (REJECT, stale_epoch)
  BatchQueue.push((S, packet, next_layer))

When BatchQueue.size() >= B_min OR BatchWindow expires:
  batch := BatchQueue.drain() // collect all queued packets
  batch := PadToMinSize(batch, B_min)  // pad with cover if needed

  // IDEAL: leak only |batch| (size) and epoch to adversary.
  // Leak(|batch|, epoch) to A.
  // A learns NOTHING else about sender->output mapping.

  sigma := UniformRandom(Permutations(|batch|))  // secret permutation
  for i in 0..|batch|:
    output[i] := SphinxProcess(batch[sigma(i)])  // one-layer decryption

  proof := BayerGrothProve(C_in, C_out, sigma, CRS)  // ZK shuffle proof
  ProofStore[BatchId] := proof

  // Deliver output packets in permuted order to next_layer nodes.
  // Proof is published to DHT; any verifier can check it.
  ForwardAll(output, next_layer)
  PublishToDHT(BatchId, C_in, C_out, proof)

On (VERIFY, BatchId) from any verifier V:
  proof := ProofStore[BatchId]
  Return BayerGrothVerify(C_in, C_out, proof, CRS)
```

6.2 Security Properties

Correctness: Every output packet is the correctly processed version of some input packet. No packet is dropped, duplicated, injected, or forwarded in input order. Formally: the output commitment C^{out} is a valid permutation of C^{in} under the ZK proof.

Zero-knowledge: The shuffle proof π reveals nothing about the permutation σ. Formally: in the simulation, the ideal functionality can replace the real prover's proof with a simulated proof

produced without knowledge of σ, indistinguishable from a real proof under the Discrete Logarithm assumption in BLS12-381.

Soundness: A cheating prover cannot produce a valid proof for an incorrect shuffle. Formally: no PPT adversary can produce (C^{in}, C^{out}, π) such that BayerGrothVerify = 1 but C^{out} is not a valid permutation of C^{in}, except with probability negl(λ) under the DL assumption in BLS12-381.

Sender anonymity from F^{MIX}: Given a batch of B packets, F^{MIX} leaks only the batch size and timing to the adversary. The adversary's view of any single sender's contribution to the batch is uniform over all B positions. Sender anonymity within a batch: 1/B.

> **Trusted Setup Dependency**
>
> The BayerGroth ZK argument requires a CRS generated in a trusted setup ceremony. If the ceremony is compromised (the trapdoor τ is known to the adversary), soundness fails: the adversary can forge proofs for any (C^{in}, C^{out}) pair, silently routing packets in any order while generating valid proofs. This is the CRIT-02 finding. Until the STARK migration is complete, F^{MIX} is only realisable under the assumption that the CRS ceremony was conducted honestly. This assumption must appear in the Trust Assumption Register (§12).

6.3 The Batch Metadata Leak

F^{MIX} leaks (|batch|, epoch, timing) to the adversary. This is the minimum necessary information leakage: a mix network that hides batch timing is not real-time. However, the real-world protocol Π^{MIX} leaks more than F^{MIX} due to the DHT publication of C^{in} and C^{out}. Specifically:

- The timing gap between C_in and C_out publication reveals how long batch processing took, which correlates with batch size and node load.
- The 8-byte C_batch field in each Sphinx⁺ packet header is visible to every hop. This is a cross-batch correlation point: a packet carrying the same C_batch value across multiple hops can be identified as being in the same batch chain (MED-03).
- The rate at which a node publishes C_in values reveals its traffic volume and uptime pattern, which is an intersection attack input (HIGH-01).

The real protocol Π^{MIX} does NOT perfectly UC-realise F^{MIX} because of these additional leaks. The gap between F^{MIX} and Π^{MIX} must be explicitly bounded in the composability analysis (§10).

7. F^{Hs} — Ideal Functionality: Hidden Service Layer

F^{HS} provides anonymous two-way communication between a client C and a server S operating at a .phantom address, in the F^{PKI}-hybrid and F^{MIX}-hybrid model. In the ideal world, the trusted party F^{HS} delivers messages between C and S without revealing either party's identity to the other or to the adversary.

7.1 Functionality Definition

```
Functionality F_HS  (in F_PKI and F_MIX hybrid model)
=====================================================

State:
  Services  : Map<PhantomAddress, ServerPublicKey>
  Sessions  : Map<SessionId, (Client, Server, SessionKey)>

On (REGISTER_SERVICE, addr, pk_ed, pk_dil) from server S:
  Verify addr = BLAKE3("phantom-hs-v1" || pk_ed || pk_dil || ...)
  Services[addr] := (pk_ed, pk_dil)
  Return (REGISTERED)

On (CONNECT, addr) from client C:
  If addr not in Services: Return (NOT_FOUND)
  session_key := UniformRandom(K)         // ideal session key
  sid := NewSessionId()
  Sessions[sid] := (C, Services[addr], session_key)
  // Leak to adversary: (SESSION_STARTED, |addr|, epoch)
  // A learns: a session to addr started. Not who C is.
  Notify S: (INCOMING_SESSION, sid)        // S learns a session exists
  Return (SESSION_ESTABLISHED, sid, session_key) to C

On (SEND, sid, message) from C or S:
  (client, server, sk) := Sessions[sid]
  ciphertext := Encrypt(sk, message)
  // Leak to adversary: (MESSAGE, |message|, epoch)
  // A learns: message size and timing. Nothing else.
  Deliver ciphertext to the other party in the session.

On (CLOSE, sid):
  Sessions.remove(sid)
```

7.2 Security Properties

Server anonymity: F^{HS} reveals the .phantom address to the client (necessary for connection). It does not reveal the server's IP address or NodeDescriptor to the client, adversary, or any relay. The real protocol realises this via the rendezvous protocol (Spec № 1 §8).

Client anonymity: F^{HS} reveals to the server only that a client connected to its address, not who the client is. No session is linkable to a specific client identity across connections.

Self-authentication: The client verifies BLAKE3(pk_ed ‖ pk_dil ‖ ...) = addr at connection time. No certificate authority is needed. Formally: no PPT adversary can produce a session that passes verification at an addr without holding the corresponding private keys.

What F^{HS} cannot guarantee: If the server is honest but foolish (e.g., leaks application-layer information that identifies it), F^{HS} provides no protection. F^{HS} guarantees transport-layer anonymity, not application-layer anonymity. The rendezvous node R learns that a session was established but not the identities of either party.

7.3 Predecessor Attack Gap in F_HS

The real protocol Π^{HS} does not perfectly realise F^{HS} in the presence of a Class-III adversary due to the predecessor attack (GAP-06). Specifically: in F^{HS}, the adversary learns only (SESSION_STARTED, |addr|, epoch). In Π^{HS}, an adversarial first-relay node observes the client's traffic entering the mix network and can correlate this with the rendezvous node's traffic to identify the session's originator over multiple sessions.

> **Open Design Decision Required**
>
> The gap between F_HS and Π_HS due to the predecessor attack requires one of three design decisions before Phase 4 implementation: (a) accept the gap and document it as an explicit protocol limitation in the threat model; (b) implement lightweight entry guards (each client maintains 2–3 long-term first-hop preferences, consistent with the Tor guard model); or (c) demonstrate that the ZK shuffle proof provides equivalent protection to entry guards — which it does not. This decision must be recorded in this specification before Phase 4 begins.

8. F^{Dcn} — Ideal Functionality: DC-Net Broadcast Layer

F^{DCN} provides information-theoretic sender anonymity within a group G of N participants. It is the strongest anonymity primitive in the protocol stack. Unlike F^{MIX}, F^{DCN} requires no computational hardness assumption for its anonymity guarantee — it holds even against a computationally unbounded adversary who observes all traffic.

8.1 Functionality Definition

```
Functionality F_DCN  (information-theoretic model)
==================================================

Parties:  Group G = {P_1, ..., P_N}, where N in [8, 128]
          Adversary A controls a subset G_A subset G, |G_A| < N.

State:
  PadMatrix  : Map<(i,j), RandomPad>  // N*(N-1)/2 pairwise pads
  RoundSlot  : Map<EpochRound, NodeId> // reservation protocol

On (BROADCAST, message m) from sender S in G:
  // Reservation phase (leaks: slot claim exists, not who claimed it)
  slot := ReservationProtocol(S, G)

  // Broadcast phase
  for each P_i in G:
    if P_i == S:
      broadcast_i := m XOR XOR_{j != i} PadMatrix[(i,j)]
    else:
      broadcast_i := XOR_{j != i} PadMatrix[(i,j)]

  XOR_result := XOR(broadcast_1, ..., broadcast_N) = m  // pads cancel

  // F_DCN leaks to adversary A:
  // - The XOR result m (it is broadcast to all)
  // - Each corrupted party P_i's broadcast value broadcast_i
  // - The group membership G (necessary for coordination)
  // F_DCN does NOT leak: which P_i is the sender.

  Deliver m to all parties in G.
```

8.2 Information-Theoretic Anonymity Proof

The DC-Net anonymity guarantee is information-theoretic, not computational. The proof is classical (Chaum, 1988):

```
Theorem: Perfect Sender Anonymity in DC-Net

Given: N participants, at most N-1 of whom are corrupted by A.
       Pairwise pads are uniformly random and unknown to A
       for at least one honest pair (P_i, P_j) where both are uncorrupted.

Then: For any two participants P_s, P_t in G \ G_A (honest participants),
      the joint distribution of A's view when P_s is the sender is
```

```
      IDENTICAL to A's view when P_t is the sender.

Proof: A observes {broadcast_i : P_i in G_A} and the XOR result m.
       The broadcast of each honest party P_i (i not in G_A) is
       broadcast_i = (if i==s: m else 0) XOR XOR_{j!=i} PadMatrix[(i,j)].
       Since A does not know PadMatrix[(i,j)] for honest pairs,
       broadcast_i is uniformly random from A's view regardless of
       whether P_i is the sender. By the one-time pad argument,
       A's view is identically distributed for any honest sender.
QED.
```

8.3 The Jamming Vulnerability and Its Formal Impact

F^{DCN} assumes honest behaviour from participants in the broadcast phase. A malicious participant who broadcasts random bits instead of the correct XOR of their pads causes the XOR result to equal m XOR noise, which is indistinguishable from a random string. The real protocol Π^{DCN} does NOT UC-realise F^{DCN} without a jamming-resistant mechanism, because a single corrupt participant in G^A can prevent any honest participant from ever successfully broadcasting.

Mitigations (see HIGH-02 in audit): the Verdict protocol (Ford et al., CCS 2012) adds a ZK proof that each participant's broadcast is the correct XOR of their shared pads. This makes jamming detectable and attributable, at the cost of adding a computational assumption (soundness of the ZK proof). With Verdict, F^{DCN} is realisable against Class-II adversaries but not against computationally unbounded Class-I adversaries — the information-theoretic guarantee is weakened to computational for the attribution check.

8.4 Pad Establishment Protocol (Open Design Problem)

The N*(N-1)/2 pairwise pads must be established before each round. The current Phantom specification does not define this protocol. Three options exist, with different security implications:

Option	Mechanism	Security	Dependency
A	Establish pads via Sphinx⁺ mix network (use F_MIX to exchange pad seeds)	Computational (bounded by Sphinx⁺ security). DC-Net's anonymity reduces to computational.	F_MIX must be realised first. F_DCN becomes a hybrid IT/computational system.
B	Out-of-band pad exchange (e.g., QR code, physical meeting)	Information-theoretic preserved. Adversary cannot learn pads.	Requires physical or side-channel contact between all N(N-1)/2 pairs. Not practical at N>8.
C	Cryptographic pseudorandom pad generation from a shared group seed (established once via F_MIX)	Computational (bounded by PRF security). Each round's pads are derived deterministically.	One-time bootstrapping via F_MIX. Subsequent rounds are independent. Most practical option.

Table 3 — Pad establishment options for DC-Net groups. Option C is recommended.

Recommendation: Adopt Option C. Use F^{MIX} to securely deliver a group seed to all N participants during group formation. Each round derives its pad matrix from BLAKE3(seed ‖

round_counter ‖ pair_id). This reduces F^{DCN}'s anonymity guarantee from information-theoretic to computational (PRF security), but preserves it against Class-II adversaries and makes the protocol practical at N=128. Document this reduction explicitly.

9. F^{cov} — Ideal Functionality: Cover Traffic Engine

F^{cov} produces a stream of cover packets that are computationally indistinguishable from real traffic packets to any observer who does not hold the decryption keys. It is the component that makes timing and volume analysis non-viable within a single epoch.

9.1 Functionality Definition

```
Functionality F_COV
====================

On (TICK) every Delta_t milliseconds (default: 100ms):
  If RealPacketQueue is non-empty:
    Emit next real packet from queue
  Else:
    Emit a COVER packet: a uniformly random 1452-byte Sphinx+ packet
    destined for a randomly selected honest node.
    // COVER packets are indistinguishable from REAL packets.
    // Cover destination nodes discard the cover payload silently.

// F_COV leaks to adversary A:
//   - The emission timing of each packet (one per Delta_t).
//   - The destination node_id of the cover packet (observable from DHT).
// F_COV does NOT leak:
//   - Whether a given packet is real or cover.
//   - The content or routing of real packets.
```

9.2 What F_COV Guarantees and Does Not Guarantee

Guarantees: The emission rate from any node is constant at 1 packet per Δt, regardless of real traffic load. An adversary observing the emission rate learns nothing about whether a node is sending real traffic at any given moment. This eliminates single-epoch timing correlation attacks.

Does not guarantee (single-epoch): F^{cov} does not hide that a node is running Phantom Protocol. The constant-rate fixed-size emission pattern is itself a fingerprint distinguishable from real HTTPS traffic (HIGH-04). The real protocol Π^{cov} must implement transport obfuscation on top of F^{cov}'s packet stream to prevent DPI-based blocking.

Does not guarantee (multi-epoch): F^{cov} operates per-epoch. It does not prevent intersection attacks across epochs: a GPA observing which nodes are online during each epoch when a specific message appears can narrow the sender set over time, independently of cover traffic content. Cover traffic hides *what* is sent in an epoch, not *whether* a node is online.

> **The Fundamental Cover Traffic Limitation**
>
> Cover traffic prevents single-epoch correlation but cannot prevent long-term intersection attacks. These are orthogonal attack vectors. F^{cov} successfully eliminates Class-I single-epoch timing analysis. It does not address Class-III long-term observation. The multi-epoch anonymity bound in §11 quantifies what remains after F^{cov} has done its work.

9.3 Interaction with F_MIX: The Proof-Timing Side Channel

This is the most important interaction between subsystems in the Phantom stack, and it is not addressed in any prior specification. When a mix node is generating a ZK shuffle proof (step 7 of the batch lifecycle), its CPU load increases significantly. Under the current design, F^{cov} continues to emit packets at the constant rate Δt during proof generation. However, the *scheduling jitter* of those packets may increase slightly due to CPU contention.

This creates a timing side channel: an adversary monitoring packet emission timing at microsecond resolution can detect when a node is generating a proof, and can correlate this with the DHT publication of C_in (which reveals batch start time). This narrows the batch timing oracle attack (MED-01) from the 200ms collection window to the actual proof generation time (~300ms single-core, ~30ms parallelised).

Mitigation: Isolate proof generation into a dedicated CPU core that does not share a scheduling queue with the packet emission thread. On modern multi-core hardware, this is straightforward. The implementation must pin the F^{cov} emission thread and the proof generation thread to separate physical cores and document this as a deployment requirement for Full-tier nodes.

10. Subsystem Composability — The Three Interaction Leaks

The UC composition theorem guarantees that if each subsystem UC-realises its ideal functionality, the full stack is secure. However, there are three points where the real protocols deviate from their ideal functionalities in ways that create cross-subsystem information leaks. These are the composability gaps that the audit identified as the most serious structural risk.

10.1 Interaction Leak 1: DHT Metadata Creates an Intersection Attack Surface

F_PKI ↔ F_MIX: DHT Publication Timing

What happens: F^{MIX} requires publishing C^{in} and C^{out} to the DHT (operated by Π^{PKI}). These publications are timestamped and observable by any DHT participant, including adversarial nodes.

What leaks: The timing and frequency of C^{in} publications for each node reveals its batch rate (and therefore its traffic volume), its uptime pattern (and therefore its intersection attack surface), and the correlation between nodes that publish C^{in} values in overlapping time windows (indicating they are handling the same generation of packets).

Formal gap: In the ideal world, F^{MIX} leaks only (|batch|, epoch) to the adversary. In the real world, Π^{MIX} additionally leaks (publish_timestamp, batch_rate, uptime_pattern) via the DHT. This additional leakage is not modelled in any prior specification and is not accounted for in the anonymity bound.

Mitigation: Add ±50ms random jitter to C^{in} publication timing (MED-01). This reduces the temporal precision of the leak from ~1ms (DHT timestamp resolution) to ~100ms, significantly degrading the intersection attack correlation. Additionally, encrypt the C_batch header field per-hop (MED-03).

10.2 Interaction Leak 2: Cover Traffic Load Reveals Proof Generation

F_MIX ↔ F_COV: CPU Scheduling Correlation

What happens: F^{MIX}'s proof generation is computationally intensive. F^{COV}'s packet emission depends on CPU scheduling. When both run in the same OS scheduling domain, proof generation increases scheduling latency for emission packets.

What leaks: Microsecond-level timing jitter in F^{COV}'s output stream correlates with the start and end of proof generation, allowing an adversary to reconstruct batch timing more precisely than the DHT timestamps alone.

Formal gap: F^{COV} is defined to emit at exactly Δt intervals. Π^{COV} emits at $\Delta t + \varepsilon(t)$, where $\varepsilon(t)$ is a scheduling-dependent noise term that is correlated with Π^{MIX}'s proof computation load. This correlation is not present in the ideal model.

Mitigation: CPU core pinning (see §9.3). Pin F^{COV} emission to a dedicated real-time-priority OS thread on a separate physical core from the proof generation thread. This reduces $\varepsilon(t)$ to hardware interrupt jitter (~1μs), which is below the adversary's practical measurement resolution for network packets.

10.3 Interaction Leak 3: DC-Net Pad Exchange Reduces Anonymity to Computational

F_DCN ↔ F_MIX: Pad Establishment Security Reduction

What happens: DC-Net pads are established via F^{MIX} (recommended Option C from §8.4). The pad seeds are delivered through the mix network as Sphinx$^+$ packets.

What leaks: If Π^{MIX} is compromised (e.g., by a Class-II adversary controlling all hops of the pad-delivery circuit), the adversary can learn the pad seeds, deriving all future DC-Net pads for that group. This collapses DC-Net's information-theoretic anonymity to the computational security of Π^{MIX}.

Formal gap: F^{DCN} provides information-theoretic anonymity unconditionally. Π^{DCN} with Option C pad establishment provides computational anonymity only, bounded by the security of Π^{MIX}. Users who choose DC-Net mode expecting information-theoretic security receive computational security without explicit notification.

Mitigation: (1) Explicitly document in the user interface that DC-Net mode provides information-theoretic anonymity only when pads are established out-of-band (Option B). For in-band pad establishment (Option C), document the security as computational. (2) For high-security use cases, provide a manual pad entry mechanism that allows Option B for small groups ($N \leq 8$). (3) The specification must state prominently: 'DC-Net mode over Phantom-established pads provides computational, not information-theoretic, anonymity.'

11. Quantified Anonymity Bounds

The whitepaper states sender anonymity as '1/k where k is the anonymity set size.' This section replaces that informal statement with quantified bounds for each protocol mode and adversary class, derived from the formal model.

11.1 Per-Epoch Anonymity in Standard Mix Mode

```
Per-epoch anonymity bound (standard mix mode, Class-I and Class-II adversary):

Given:
  B   = batch size (packets per batch, default B = 256)
  L   = circuit length (hops, default L = 3)
  f   = fraction of adversarial nodes (f < 1/2)
  k_b = min(B, stratum_size) = effective anonymity set within one batch

Sender deanonymisation probability in one epoch:

  Against Class-I (passive): epsilon_I = 0
  (Cover traffic hides whether real traffic was sent in any epoch)

  Against Class-II (f fraction of nodes controlled):
  epsilon_II = Pr[all L hops controlled] = f^L
  For f = 0.2, L = 3: epsilon_II = 0.008  (0.8%)
  For f = 0.3, L = 3: epsilon_II = 0.027  (2.7%)

  Note: ZK shuffle proof prevents deanonymisation even when hops are
  controlled, UNLESS the adversary controls all L hops AND the CRS
  ceremony was compromised. If CRS is compromised: epsilon_II = 1.
```

11.2 Multi-Epoch Anonymity Degradation

```
Multi-epoch anonymity degradation (intersection attack, Class-III):

Given:
  T     = number of epochs observed
  p_on  = probability a sender is online in any given epoch
  N     = total network size
  k_0   = initial anonymity set (senders who could be observed in epoch 0)

After T epochs, expected anonymity set size:
  k(T) = N * (p_on)^T

For a full-time node (p_on = 1.0): k(T) = N  (no degradation)
For a mobile node (p_on = 0.1):
  T=1:   k(1) = N * 0.1   = 0.1N
  T=10:  k(10) = N * 10^{-10} ≈ 0 (fully deanonymised for N < 10^{10})

For the predecessor attack (Class-III, f fraction adversarial nodes):
  Pr[adversary was first hop at least once in T circuits]
  = 1 - (1 - f)^T
  For f = 0.2, T = 50:  Pr = 0.99999  (~100%)
  For f = 0.2, T = 5:   Pr = 0.672    (67.2%)
```

```
Practical implication: a user who builds 5 or more circuits is unlikely
to have avoided an adversarial first hop. Entry guards would reduce this
to Pr[adversary controls one of 2 guards] = 1-(1-f)^2 = 0.36 (constant).
```

11.3 Minimum Viable Anonymity Set k_min

k^{min} is the minimum network size below which the protocol provides insufficient anonymity to be safe for high-risk users. The definition is security-goal dependent:

Use case	Required k	Reasoning	k_min at f=0.2
General privacy (web browsing)	k ≥ 50	1/50 = 2% deanonymisation probability acceptable for low-risk users	k_min = 250 nodes
Journalist source protection	k ≥ 1,000	1/1000 = 0.1% deanonymisation per epoch acceptable	k_min = 5,000 nodes
Whistleblower / dissident	k ≥ 10,000	1/10,000 = 0.01% per epoch acceptable for high-risk users	k_min = 50,000 nodes
DC-Net extreme mode	k_group ≥ 8	Information-theoretic: any group of 8+ honest members is sufficient	8 honest group members

Table 4 — Minimum viable anonymity set by use case. k_min at f=0.2 assumes the 1/f safety margin.

Protocol action: The protocol must implement a k^{min} check at startup and before each circuit build. If the node count in the DHT falls below the k^{min} threshold for the selected use case, the protocol should issue an explicit warning: 'Network size insufficient for [use case] anonymity. Current k = [N]. Required k^{min} = [threshold]. Continue with reduced anonymity? [Y/N]' Routing should never proceed silently below k^{min}.

11.4 DC-Net Mode Anonymity

```
DC-Net anonymity (information-theoretic, Class-I and Class-II):

Given: Group G = {P_1, ..., P_N}, |G_A| = m corrupted members (m < N)

Sender anonymity set: N - m  (all honest members are equally likely senders)
Sender deanonymisation probability: 1 / (N - m)

For N = 8, m = 1: anonymity set = 7, probability = 14.3%
For N = 64, m = 0: anonymity set = 64, probability = 1.6%
For N = 128, m = 10: anonymity set = 118, probability = 0.85%

NOTE: This bound assumes Option B (out-of-band) pad establishment.
With Option C (in-band via F_MIX), the bound degrades to:
  epsilon_DCN_C = max(epsilon_DCN_B, f^L_pad)
  where L_pad is the circuit length used for pad delivery.
The information-theoretic guarantee is NOT preserved under Option C.
```

12. Trust Assumption Register

Every security guarantee in this specification rests on one or more trust assumptions. This register makes all assumptions explicit. A guarantee is only as strong as its weakest assumption. Assumptions marked UNRESOLVED are currently unverified and must be resolved before the Phase 5 audit gate.

ID	Assumption	Used By	Status	If Violated
TA-01	BLAKE3 is a collision-resistant hash function and a random oracle (ROM assumption)	F_PKI, F_MIX, F_HS, address derivation	STANDARD — widely accepted	Node identity forgery, commitment collisions
TA-02	Dilithium-3 (ML-DSA) is EUF-CMA secure against quantum adversaries (Class-IV)	F_PKI node identity, F_HS service identity, descriptor signing	STANDARD — NIST FIPS 204	Identity forgery, descriptor spoofing
TA-03	Kyber-1024 (ML-KEM) is IND-CCA2 secure against quantum adversaries (Class-IV)	F_MIX per-hop key exchange, F_HS rendezvous KEM	STANDARD — NIST FIPS 203	Session key recovery by quantum adversary
TA-04	The Kyber-1024 96-byte fragment scheme (Spec № 2 §3.4) preserves IND-CCA2 security	F_MIX per-hop KEM decapsulation	UNRESOLVED — CRIT-01	Post-quantum secrecy collapses to X25519 layer only
TA-05	The BLS12-381 CRS was generated in an honest MPC ceremony with at least one honest participant destroying their trapdoor	F_MIX ZK shuffle proof soundness	UNRESOLVED — CRIT-02. Ceremony not yet conducted.	Proof forgery. Any adversary can fake shuffle proofs. Anonymity collapses silently.
TA-06	Discrete Logarithm in BLS12-381 is hard for classical and quantum adversaries	F_MIX Pedersen commitments, Bayer-Groth soundness	STANDARD — BLS12-381 is quantum-vulnerable (Shor's). Mitigated by STARK migration roadmap.	Commitment opening, proof forgery if CRQC exists
TA-07	The fraction of adversarially controlled nodes f < 1/3 at all times	Eclipse resistance (d=5, quorum=3), circuit anonymity	OPERATIONAL ASSUMPTION — not provable, must be maintained	Eclipse attacks become feasible; circuit anonymity collapses
TA-08	No adversary compromises all N*(N-1)/2 DC-Net pairwise pads for any group (Option C: no adversary controls all L hops of all pad delivery circuits)	F_DCN anonymity	Depends on f < 1/3 for Option C. Unconditional for Option B.	DC-Net sender anonymity collapses

ID	Assumption	Used By	Status	If Violated
TA-09	The ASN-to-IP mapping used for introduction point AS-diversity selection is accurate and not BGP-hijacked	F_HS introduction point selection diversity	UNRESOLVED — GAP-02. No data source specified.	Introduction points may be co-located under adversary control despite apparent AS diversity
TA-10	The network reaches k ≥ k_min before sensitive use begins	All anonymity bounds in §11	OPERATIONAL — protocol must enforce at runtime	Anonymity set too small; statistical deanonymisation trivial
TA-11	ChaCha20-Poly1305 provides IND-CPA and INT-CTXT security under standard assumptions	Session payload encryption, F_MIX payload layer	STANDARD — universally accepted	Session payload confidentiality and integrity
TA-12	Ed25519 is EUF-CMA secure against classical adversaries (Class-I, Class-II)	F_PKI descriptor signing (classical layer)	STANDARD — accepted for classical adversaries. Vulnerable to Class-IV (Shor).	Classical: no impact. Quantum: identity forgery. Mitigated by Dilithium-3 layer.

Table 5 — Trust assumption register. UNRESOLVED items must be resolved before Phase 5 audit gate.

13. Guarantee Failure Conditions

This section states precisely the conditions under which each major security guarantee fails. This is not defeatist — it is a necessary part of an honest security specification. A protocol that does not state its failure conditions cannot be correctly deployed.

Guarantee	Holds under	Fails if	Failure is silent?
Sender anonymity (per-epoch)	TA-01, TA-07, k ≥ k_min, cover traffic active	k < k_min OR f ≥ 1/3 OR CRS compromised (TA-05)	Yes for CRS compromise. No for k, f violations (detectable by node count).
Session confidentiality	TA-01, TA-02, TA-03, TA-04, TA-11	Kyber fragment broken (TA-04) OR CRS compromised and DL broken (TA-05, TA-06)	Yes for TA-04. Yes for TA-05+TA-06 together.
Forward secrecy	Per-epoch KEM key deletion after epoch end	Epoch KEM keys not deleted (implementation bug) OR CRQC + Kyber broken (TA-03+TA-04)	Yes (implementation bug is silent)
ZK shuffle proof soundness	TA-05, TA-06	CRS trapdoor known to adversary (TA-05 violated)	YES. A compromised CRS allows undetectable proof forgery. This is the most dangerous silent failure mode.
DHT eclipse resistance	TA-07 (f < 1/3), d=5 quorum=3	f ≥ 1/3 sustained Sybil attack	Partial — eclipsed lookups appear normal to victim
DC-Net info-theoretic anonymity	Option B pad establishment, m < N corrupted group members	Option C pad establishment (reduces to computational) OR adversary controls pad delivery circuit	YES under Option C — user is not notified of security reduction
Long-term (multi-epoch) anonymity	Low uptime variance, entry guards, T << 1/f circuits	No entry guards + T ≥ 5 circuits + f ≥ 0.2 (predecessor attack)	Yes — no protocol mechanism detects or warns about predecessor attack
Hidden service server anonymity	TA-07, introduction point AS diversity, f < 1/3	All N_INTRO introduction points compromised OR BGP hijack of ASN data (TA-09)	Partial — service becomes unreachable if IPs all compromised,

Guarantee	Holds under	Fails if	Failure is silent?
			not covertly deanonymised
Minimum anonymity set enforcement	Protocol checks k ≥ k_min before routing	k_min check not implemented (implementation omission)	YES if not implemented — users may route through a tiny network believing full guarantees hold

Table 6 — Guarantee failure conditions. Silent failures are the highest priority to mitigate.

14. Open Assumptions and Research Questions

The following are genuine open problems that this specification cannot resolve. They require academic research, cryptographic review, or design decisions that have not yet been made. Each is tracked here to ensure they are not forgotten between specification versions.

14.1 Can the Kyber Fragment Scheme Be Proven Secure? (TA-04)

The 96-byte Kyber fragment scheme (Spec № 2 §3.4) is the most important open cryptographic question in the protocol. There are two possible resolutions: (a) a security proof that shows the fragment scheme preserves IND-CCA2 under the assumptions of FIPS 203, or (b) a proof of insecurity that rules out the construction. Either resolution is acceptable — if insecure, the alternative is to use the Outfox-style pre-encapsulated full ciphertext approach. What is not acceptable is deploying the scheme without resolution.

Responsible party: External cryptographer (to be engaged before Phase 2 begins). Target: resolution by month 4 of the project timeline.

14.2 Does the Multi-Epoch Anonymity Degradation Have a Lower Bound?

The intersection attack degradation formula in §11.2 gives k(T) = N * (p_on)^T. This gives a practical upper bound on anonymity. The question is whether cover traffic provides any lower bound on the degradation rate — i.e., can cover traffic be shown to make p_on appear artificially high to an adversary? If so, by how much? This is an open research question.

Known partial result: Danezis (2003) shows that cover traffic does not prevent statistical disclosure attacks by a Class-III adversary. The cover traffic rate needs to be substantially higher than the real traffic rate to provide any meaningful protection, and even then the protection is not absolute. This is consistent with our claim that cover traffic eliminates Class-I timing correlation but does not address Class-III intersection attacks.

14.3 What Is the Security Model for the STARK Migration?

The roadmap targets replacing Groth16 with a STARK-based proof system (eliminating the trusted setup). The security model for the replacement proof system must be compatible with the UC framework defined in this specification. Specifically: the replacement must provide (a) computational soundness under a falsifiable assumption (e.g., hash function security in the ROM), (b) zero-knowledge in the QROM, and (c) proof generation time within the 300ms budget for N=256 packets.

Candidate systems: Plonky2 (Polygon Zero) — recursive SNARKs, Poseidon-based, no trusted setup. Boojum (zkSync) — STARK-based, Rust-native. Winterfell (Facebook) — STARK library in Rust. Of these, Boojum and Winterfell are Rust-native and compatible with the Tokio async runtime. Plonky2 has the most production deployments. None have been evaluated for the specific Bayer-Groth-equivalent shuffle argument.

14.4 Can Entry Guards Be Added Without Violating the True P2P Design Goal?

The predecessor attack (GAP-06) is provably mitigated by entry guards. The design goal of 'every client is a routing node' is architecturally incompatible with traditional entry guards. A lightweight hybrid is possible: each node maintains a preference set of 2–3 first-hop relays,

chosen for long uptime and diverse ASN, that are used preferentially for the first hop of each circuit. This does not eliminate the client/relay distinction — every node still routes for others. It merely biases first-hop selection toward stable, verified nodes.

Design question: Does this hybrid approach provide the predecessor attack protection of entry guards while preserving the anonymity set benefit of true P2P? A formal analysis is needed. The answer is likely yes for Class-II adversaries and partially yes for Class-III.

14.5 What Is the Correct DC-Net Pad Establishment Protocol?

This specification recommends Option C (PRF-based derivation from a mix-network-delivered seed) as the most practical approach. However, this recommendation reduces DC-Net's information-theoretic guarantee to computational. A better solution — one that preserves the information-theoretic bound while remaining practical at N=128 — has not been identified. This is a genuine open research problem in the DC-Net literature. Hierarchical DC-Nets (where smaller subgroups have out-of-band pads and the hierarchy aggregates) may be a viable path.

15. Relationship to Other Specifications

Specification	Relationship to This Document	Key Constraints This Spec Imposes
Whitepaper v0.1	Provides informal threat model and design goals. This spec formalises and in some cases corrects those goals.	The '1/k anonymity guarantee' must be replaced by the quantified bounds in §11. Long-term anonymity against Class-III is not guaranteed.
Spec № 1: Hidden Services	Defines the rendezvous protocol that Π_HS implements. F_HS in this spec is the ideal version.	Must add: (a) predecessor attack gap documentation; (b) entry guard design decision; (c) ASN data source specification (TA-09); (d) k_min check before circuit build.
Spec № 2: Sphinx⁺ and ZK Shuffle	Defines the packet format and proof system that Π_MIX implements. F_MIX in this spec is the ideal version.	Must add: (a) Kyber fragment security proof or replacement (TA-04); (b) C_batch encryption per-hop; (c) proof generation thread isolation requirement; (d) timing jitter for C_in publication.
Spec № 3: DPKI / Kademlia DHT	Defines the node directory that Π_PKI implements. F_PKI in this spec is the ideal version.	Must add: (a) k_min enforcement mechanism; (b) BGP-hijack-resistant ASN data source; (c) Argon2id replacement for PoW static puzzle; (d) C_batch expansion to 16 bytes.
Security Audit (March 2026)	Identified the non-composability gap (GAP-04) that this spec addresses. All CRIT and HIGH findings referenced here.	This spec formally captures GAP-04. CRIT-01 and CRIT-02 are captured in TA-04 and TA-05. All unresolved assumptions must be resolved before Phase 5.
Cross-Assessment (March 2026)	Confirmed 7 genuine gaps. GAP-01 through GAP-07 are addressed in this spec.	k_min (§11.3) addresses GAP-01. TA-09 addresses GAP-02. Leak 3 (§10.3) addresses GAP-03. This spec is the resolution of GAP-04. §8.3 addresses GAP-05. §7.3 addresses GAP-06. TA-04 + §14.1 address GAP-07.

Table 7 — Cross-specification dependency and constraint map.

16. References

[Can01] Canetti, R. — Universally Composable Security: A New Paradigm for Cryptographic Protocols. FOCS 2001.

[Can07] Canetti, R., Dodis, Y., Pass, R., Walfish, S. — Universally Composable Security with Global Setup. TCC 2007.

[BPW04] Backes, M., Pfitzmann, B., Waidner, M. — A Universally Composable Cryptographic Library. IACR ePrint 2003/015.

[Unr10] Unruh, D. — Universally Composable Quantum Multi-Party Computation. EUROCRYPT 2010.

[Cha88] Chaum, D. — The Dining Cryptographers Problem: Unconditional Sender and Recipient Untraceability. Journal of Cryptology 1(1), 1988.

[DG09] Danezis, G., Goldberg, I. — Sphinx: A Compact and Provably Secure Mix Format. IEEE S&P 2009.

[BG12] Bayer, S., Groth, J. — Efficient Zero-Knowledge Argument for Correctness of a Shuffle. EUROCRYPT 2012.

[VER12] Ford, B. et al. — Verdict: Privacy-Preserving Group Accountability. CCS 2012.

[COR10] Corrigan-Gibbs, H., Ford, B. — Dissent: Accountable Anonymous Group Messaging. CCS 2010.

[WRI04] Wright, M., Adler, M., Levine, B., Shields, C. — The Predecessor Attack. ACM TISSEC 2004.

[DAN03] Danezis, G. — Statistical Disclosure Attacks: Traffic Confirmation in Open Environments. IFIP SEC 2003.

[MIX24] Oldenburg et al. — MixMatch: Flow Matching for Mixnet Traffic. PoPETs 2024(2).

[INR25] Netto, Cholez, Ignat — Active Sybil Attack in IPFS DHT. Inria hal-05424411, May 2025.

[KYB24] Bernstein et al. — KyberSlash. IACR ePrint 2024/1049.

[OFX24] Nym Technologies — Outfox: A Post-Quantum Packet Format for Layered Mixnets. arXiv:2412.19937, 2024.

[NIST203] NIST — ML-KEM (CRYSTALS-Kyber), FIPS 203. August 2024.

[NIST204] NIST — ML-DSA (CRYSTALS-Dilithium), FIPS 204. August 2024.

[RFC9180] Barnes et al. — Hybrid Public Key Encryption. IETF RFC 9180, 2022.

[BM07] Baumgart, Mies — S/Kademlia: A Practicable Approach Towards Secure Key-Based Routing. IEEE ICPADS 2007.

[PHS1] Phantom Protocol — Technical Specification № 1: Hidden Services. March 2026.

[PHS2] Phantom Protocol — Technical Specification № 2: Sphinx$^+$ and ZK Verifiable Shuffle. March 2026.

[PHS3] Phantom Protocol — Technical Specification № 3: DHT-Based Trustless PKI. March 2026.

[AUDIT] Phantom Protocol — Security Architecture Audit. March 2026.

[CROSS] Phantom Protocol — Cross-Assessment Comparative Analysis. March 2026.

— Phantom Protocol Technical Specification № 4 — End of Document —